

Contract Clause Extraction for Operational Risk Review: A Comparative Study of Sliding-Window and TF-IDF Retrieval-Assisted Legal-BERT Fine-Tuning on the CUAD Dataset

1st Abhishek Gowda

Department of Business

University of Europe for Applied Sciences

abhishekgowdat05@gmail.com

2nd Shirisha Shashidhar Reddy

Department of Business

University of Europe for Applied Sciences

shirisha.shashishar@ue-germany.de

3rd Manya Megharaj

Department of Business

University of Europe for Applied Sciences

manya.megharaj@ue-germany.de

Abstract—Reviewing a commercial contract for risk is a slow, manual job: a lawyer reads a long and largely unstructured document looking for the handful of clauses that actually matter, such as termination rights, governing law, or a cap on liability. This paper describes an applied machine learning system that flags and locates those clauses automatically, using the Contract Understanding Atticus Dataset (CUAD). Contracts are almost always longer than a transformer encoder can read in a single pass — 97.2% of the examples in our selected clause set exceed the 512-token limit of Legal-BERT — so we treat clause detection as a chunk-level binary classification problem combined with span extraction, and we compare two ways of deciding which chunks to score. Approach A is an exhaustive sliding window that runs the model over every 512-token window of a contract. Approach B first uses TF-IDF to rank the contract’s own chunks against the clause query and only sends the five most similar chunks to the transformer. Across eight clause types and 816 held-out test examples, Approach B beats Approach A on every single clause, lifting mean test F1 from 0.480 to 0.688 and mean AUPR from 0.582 to 0.788, while also doing far less transformer work because low-relevance chunks are discarded before the model ever sees them. We additionally report span-level results (token F1 = 0.510, exact match = 0.125) and a systematic error analysis, which shows that the cases we miss cluster in clauses with low positive prevalence and long, narratively embedded answer spans. We argue that the improvement is a retrieval effect rather than a modelling effect, and we discuss where the system still falls short in practice.

Keywords—Legal NLP, contract review, CUAD, Legal-BERT, clause extraction, information retrieval, TF-IDF, span extraction, class imbalance.

I. INTRODUCTION

When a lawyer reviews a commercial contract for operational risk, they are really answering the same small set of questions over and over. Can either side terminate for convenience? Whose law governs the agreement? Does the counterparty get audit rights? Is there a cap on liability, or is it open-ended? The problem is that the answers are buried inside documents that can run to tens of thousands of words, and reading every contract line by line simply does not scale. The cost of a mistake is also lopsided: missing a clause (an uncapped liability, a hidden non-compete) is far more damaging than raising a flag that turns out to be nothing.

That asymmetry is what motivates the project. We want a system that flags and locates risk clauses so that a reviewer can narrow their attention, not one that tries to replace the reviewer’s judgement.

This is not plain document classification, and three properties of the task end up driving most of the design decisions. First, contracts are much longer than a transformer’s context window, so every method has to decide which part of the document to actually look at. Second, clause presence is heavily imbalanced — once a contract is split into chunks, as few as 1–4% of those chunks are positive — so a model can score very high on accuracy while never once firing on a real clause. Third, there are two outputs rather than one: whether the clause is present at all, and, if it is, exactly which sentence or sentences make it up. That means we need both a classification metric and a span-extraction metric, not a single headline number.

II. RELATED WORK

CUAD was released as an expert-annotated legal QA benchmark covering 510 commercial contracts and 41 clause categories, set up as an extractive question-answering task [1]. The underlying encoder architecture, the transformer [2], reads a fixed-length context window, and the encoders most commonly used for this kind of task — BERT [3] and its successors RoBERTa [4] and DeBERTa [5] — all cap their input at 512 tokens. Most existing CUAD baselines take one of these general-purpose or domain-adapted encoders and fine-tune it directly on contract text that has been truncated or windowed. We use Legal-BERT [6] as the encoder instead. Legal-BERT was pretrained on legal corpora, and our working assumption is that this makes it more sensitive to the boilerplate phrasing that clauses tend to use than a general-domain encoder would be.

One way to avoid chunking altogether is to widen the context window itself. Longformer [7] and Big Bird [8] use sparse attention patterns to process sequences of several thousand tokens in a single pass. We keep the standard 512-token encoder here and handle document length through windowing, which keeps the model cheap and lets us study the chunk-selection question directly. Putting a cheap lexical retriever

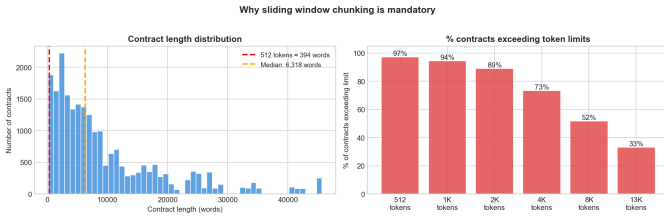


Fig. 1. Contract length distribution, confirming the chunking requirement.

such as TF-IDF or BM25 [9] in front of an expensive neural reader is also not new: it is the familiar retrieve-then-read pattern from open-domain QA, used in DrQA [10] and later in dense form by DPR [11]. The small twist here is that we apply that pattern inside a single long document rather than across a whole corpus of documents.

Contract review has also been framed in ways other than extractive QA. ContractNLI [12] casts clause review as document-level natural language inference, and LexGLUE [13] gathers a broader suite of legal language understanding tasks, several of which involve contracts. These share our motivation — cutting down the manual burden of contract review — but target classification or entailment rather than span-level clause extraction.

III. PROBLEM STATEMENT

Given a contract C and a clause type q (one of eight that we selected), the system has to decide two things: (i) whether C contains a clause of type q at all, and (ii) if it does, which exact span of text constitutes that clause. We picked the eight clause types to cover a spread of both prevalence and practical importance: Termination for Convenience, Governing Law, Audit Rights, Change of Control, Cap on Liability, License Grant, Non-Compete, and Warranty Duration. A workable solution has to cope with documents that run to tens of thousands of words without truncating the answer away, it must not collapse into a majority-class predictor when the classes are this imbalanced, and it needs to report results per clause and not only in aggregate, because clause difficulty varies a lot.

IV. DATASET

We use the official CUAD release (408 training contracts / 22,450 QA pairs; 102 test contracts / 4,182 QA pairs; 41 clause types), pulled directly from the Atticus Project GitHub repository. A few properties stood out during exploratory data analysis (Fig. 1, Fig. 2) and shaped the rest of the pipeline:

- **Contract length is very skewed.** The median contract is 6,318 words and the longest is 45,650, against Legal-BERT’s 512-token input limit. In practice 97.2% of examples go over that limit, so chunking is not optional.
- **Clause prevalence is uneven.** After filtering down to the eight selected clause types, the flattened dataset contains 4,749 training and 816 test document-level QA rows. At the document level this is roughly balanced (49.8% positive), but prevalence ranges widely

by clause, from 31% for Warranty Duration up to 87% for Governing Law (Fig. 3).

- **Answer spans are long.** They have a median of 42 words and a maximum of 479, so unlike the short single-token answers common in extractive QA, span extraction here has to handle genuinely long targets.

The real imbalance only becomes visible once the contracts are chunked into overlapping 512-token windows (Section V). Of the 211,041 training chunks, only 3,388 (1.6%) actually contain an answer span — a positive-to-negative ratio of about 1:61.

V. METHODOLOGY

A. Chunking

Every contract is split into overlapping windows of 512 tokens with a stride of 128, using the Legal-BERT tokenizer. A chunk is labelled positive if the gold answer span falls inside it. Across the 4,749 training rows and 816 test rows, this produces 211,041 training chunks and 23,639 test chunks.

B. Class-imbalance mitigation

Training on the raw 1:61 ratio is asking for a model that simply learns to say “no clause” every time. To avoid that, we undersample the negative chunks while keeping all 3,388 positives, aiming for a 3:1 negative-to-positive ratio. That leaves a balanced training set of 13,552 chunks. We deliberately leave the test set at its natural distribution, so the numbers we report reflect what the system would actually face in use.

C. Model

We fine-tune `nlpaueb/legal-bert-base-uncased` with two heads on top of the shared encoder: a binary classification head (is the clause in this chunk, yes or no) and a QA-style span head that predicts start and end token positions, in the SQuAD-style extractive setup [14]. Both heads are trained jointly for 2 epochs, with a batch size of 8 and a learning rate of 2×10^{-5} , on a single NVIDIA RTX 5050 (8.5 GB VRAM). We fixed the random seed to 42 so the run is reproducible. Fig. 4 shows the training loss and accuracy across the two epochs.

D. Approach A: Exhaustive Sliding Window

In Approach A, every chunk of a test contract is passed through the fine-tuned model. The contract is predicted positive for a clause if any of its chunks fires positive, and the reported span is taken from the highest-confidence positive chunk. This is the naive but complete baseline: nothing is ever filtered out, so recall is limited only by the model itself — but the price is that we score every window of every contract.

E. Approach B: TF-IDF Retrieval-Assisted Chunking

Approach B adds a retrieval step before the transformer. For each contract we build a TF-IDF vector space over that contract’s own chunks and score each chunk by cosine similarity to the clause query (for example, “*Highlight the parts of this contract related to Governing Law...*”). Only

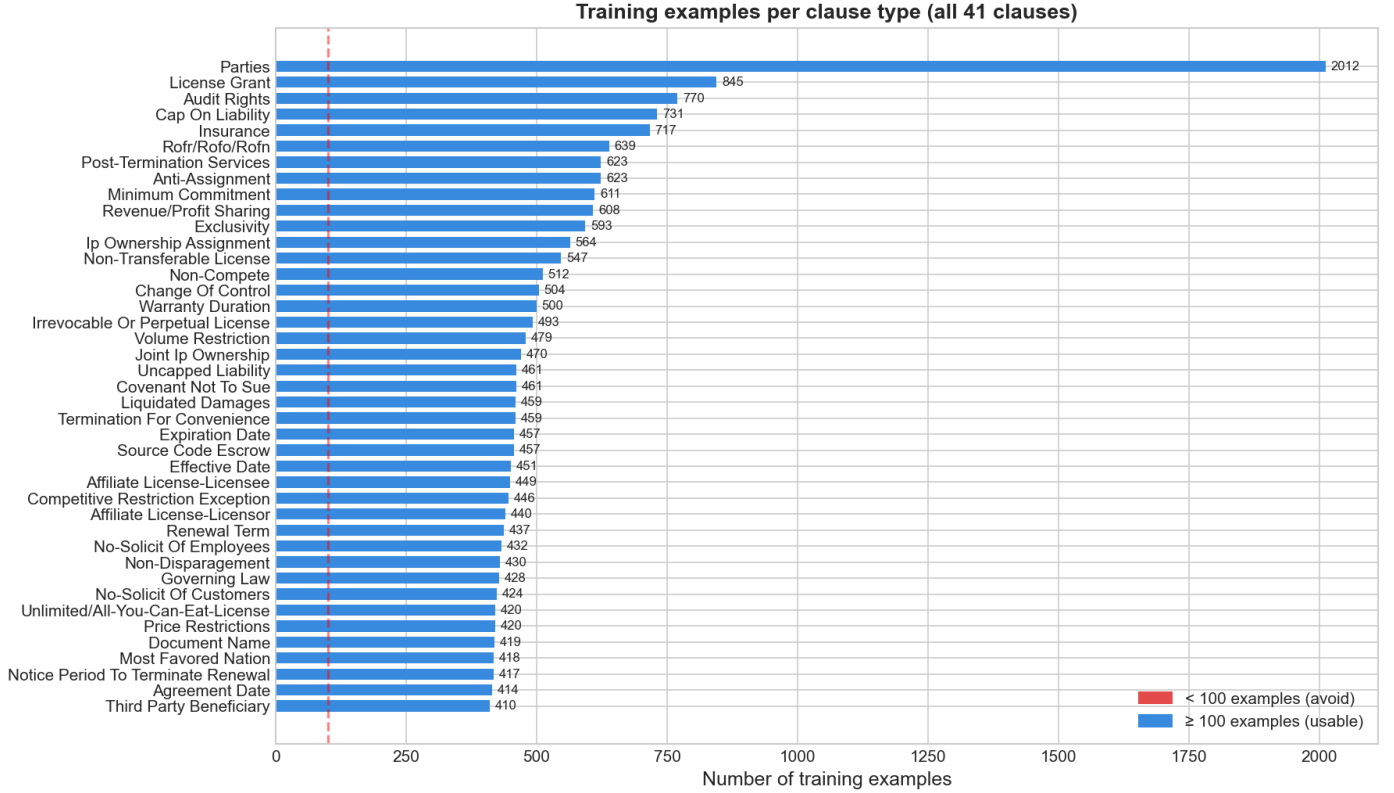


Fig. 2. Training example counts per clause type across all 41 CUAD categories.



Fig. 3. Positive vs. negative balance for the eight selected clauses.

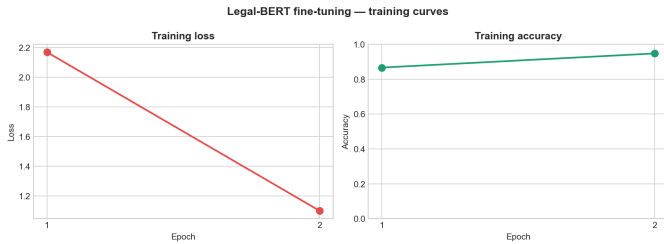


Fig. 4. Training loss and accuracy over 2 fine-tuning epochs (Approach A).

the top-5 chunks by similarity are sent to the fine-tuned model. The reasoning is that lexical retrieval should concentrate the transformer on the chunks that are plausibly relevant, which raises effective precision and shrinks the pool of low-relevance negatives that tend to generate false positives — and it does all of this at a fraction of Approach A’s inference cost.

F. Evaluation protocol

For the classification task we report per-clause and mean F1 and area under the precision-recall curve (AUPR), scored at the chunk level and then aggregated up to the contract level. For span extraction we report token-level F1 and exact match (EM), computed only on positive examples, as is the standard convention for extractive QA. Both approaches are evaluated on the same 102 held-out contracts, so the comparison is controlled. Because CUAD’s official split is document-disjoint, there is no subject or document leakage between the train and test splits.

VI. RESULTS

A. Classification performance

Table I gives per-clause test F1 and AUPR for both approaches. Approach B wins on all 8/8 clauses. Mean F1 rises from 0.480 (Approach A) to 0.688 (Approach B), and mean AUPR rises from 0.582 to 0.788. The largest gains land on the hardest, lowest-prevalence clauses — Change of Control (F1 0.312 → 0.635) and Non-Compete (F1 0.340 → 0.511). That fits the intuition that TF-IDF pre-filtering helps most exactly where an exhaustive scan produces the most false positives. Fig. 5 plots the same per-clause F1 and AUPR for both approaches side by side.

It is worth being honest about Approach A on its own: it shows textbook overfitting, with a training F1 of 0.932 against a test F1 of 0.510 on the binary task, a gap of 0.42. That is not surprising given only two fine-tuning epochs, a fairly

Approach A vs Approach B — Full Comparison

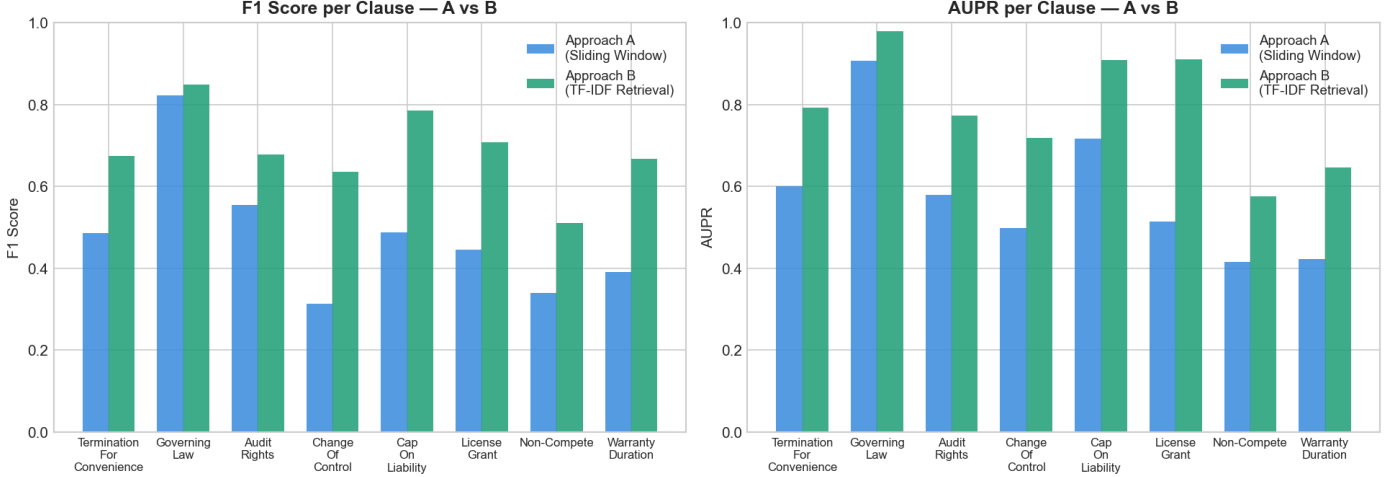


Fig. 5. Per-clause F1 and AUPR: Approach A vs. Approach B.

TABLE I. TEST-SET F1 / AUPR BY CLAUSE: APPROACH A (SLIDING WINDOW, ALL CHUNKS) VS. APPROACH B (TF-IDF TOP-5). APPROACH B IS STRONGER ON EVERY CLAUSE.

Clause	A F1	B F1	A AUPR	B AUPR
Termination f. Conv.	0.485	0.675	0.600	0.792
Governing Law	0.822	0.849	0.907	0.978
Audit Rights	0.554	0.677	0.579	0.773
Change of Control	0.312	0.635	0.498	0.718
Cap on Liability	0.487	0.786	0.717	0.909
License Grant	0.445	0.707	0.513	0.911
Non-Compete	0.340	0.511	0.416	0.576
Warranty Duration	0.390	0.667	0.422	0.646
Mean	0.480	0.688	0.582	0.788

small balanced training set, and a test distribution that is both domain-shifted and imbalanced.

B. Span extraction

On the 303 positive test contracts, Approach B reaches an overall token F1 of 0.510 and an exact match of 0.125. The low EM is mostly down to the answers being long (median 42 words) and often carrying boilerplate numbering or formatting that a normalized prediction does not reproduce verbatim; the much higher token F1 shows there is still substantial word-level overlap. Governing Law gives the best span quality (EM 0.349, token F1 0.691), because its answer spans follow a short, formulaic sentence pattern. Non-Compete and Warranty Duration are the worst (EM near 0), which lines up with their weaker classification scores and their longer, more narratively embedded phrasing.

C. Qualitative demonstration

As a concrete example, on a held-out supply agreement both approaches correctly picked out the Governing Law clause with 0.994 confidence and extracted an identical, verbatim-matching span (“it will be governed by the law of the People’s Republic of China...”). The difference was cost: Approach B only had to score 5 chunks, against 7 for

Approach A. Even when the two methods agree, retrieval-based filtering is doing measurably less work.

VII. ERROR ANALYSIS

On the full 816-row test set, Approach B produces 444 true negatives, 210 true positives, 93 false negatives, and 69 false positives (Fig. 6). False negatives outnumber false positives, which means the system currently leans towards missing clauses rather than over-flagging them. For an operational risk reviewer that is the wrong way round: a missed termination or liability clause costs more than a spurious flag that a human can dismiss in seconds.

Reading through the false negatives by hand, two causes keep coming up. The first is TF-IDF failing to surface the correct chunk when the clause is worded with vocabulary that does not overlap the query — for instance, an audit-rights clause written entirely in terms of “inspection” rather than “audit”. The second is clauses that straddle a chunk boundary, so no single 512-token window contains the whole answer. The false positives, by contrast, tend to sit in contracts with dense definitional or cross-referencing boilerplate that superficially resembles clause language.

VIII. LIMITATIONS

A few caveats are worth stating plainly. The evaluation set (816 rows, 102 contracts) is modest, and the per-clause positive counts are small (10–83 spans), so the per-clause metrics carry non-trivial variance. We only trained for two epochs because of compute constraints, and the train/test F1 gap suggests that more epochs or some regularization might improve generalization — though probably without changing whether Approach A or B comes out ahead. TF-IDF is a lexical rather than semantic matcher, and it is the main bottleneck behind our false negatives; a learned or embedding-based retriever — for example a dense passage retriever [11] or one built on Sentence-BERT embeddings [15] — would likely narrow that gap, and a wider-context encoder such as

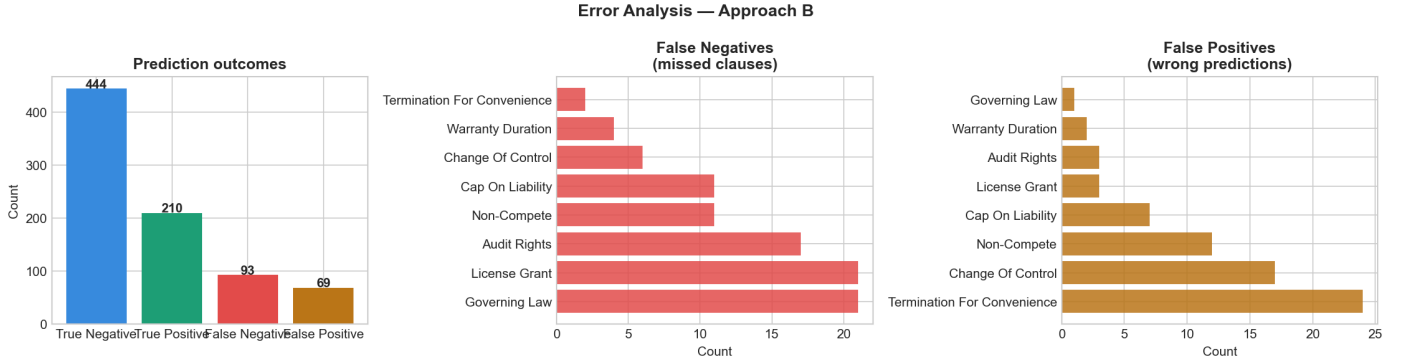


Fig. 6. Error breakdown (true/false positive/negative) for Approach B.

Longformer [7] would be another way to sidestep the chunk-boundary failures entirely. Finally, everything here is reported at the chunk and contract level for eight hand-picked clause types out of CUAD’s 41, so we cannot assume the same behaviour carries over uniformly to the categories we did not study.

IX. CONCLUSION

The question this project set out to answer was whether retrieval-based chunk filtering beats a naive exhaustive scan when a transformer’s context window is far smaller than the documents it has to read. On this dataset and these eight clause types the answer is clear: TF-IDF-assisted retrieval (Approach B) beats the exhaustive sliding-window baseline (Approach A) on every clause, both on classification quality (mean F1 +0.208, mean AUPR +0.206) and on inference cost, and it does so without touching the underlying fine-tuned Legal-BERT model. In other words, the gain comes from choosing candidates more sensibly, not from a stronger classifier — which is a useful thing to know when deploying long-document clause extraction under a compute budget. The dominant remaining error mode, false negatives driven by lexical retrieval mismatch, is the clearest next thing to work on.

REFERENCES

- [1] D. Hendrycks, C. Burns, A. Chen, and S. Ball, “CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review,” in *Proc. NeurIPS Datasets and Benchmarks Track*, 2021.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [4] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [5] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTa: Decoding-Enhanced BERT with Disentangled Attention,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.
- [6] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, “LEGAL-BERT: The Muppets straight out of Law School,” in *Findings of EMNLP*, 2020, pp. 2898–2904.
- [7] I. Beltagy, M. E. Peters, and A. Cohan, “Longformer: The Long-Document Transformer,” *arXiv preprint arXiv:2004.05150*, 2020.
- [8] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontañón, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, “Big Bird: Transformers for Longer Sequences,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [9] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [10] D. Chen, A. Fisch, J. Weston, and A. Bordes, “Reading Wikipedia to Answer Open-Domain Questions,” in *Proc. 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017, pp. 1870–1879.
- [11] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [12] Y. Koreeda and C. D. Manning, “ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts,” in *Findings of EMNLP*, 2021, pp. 1907–1919.
- [13] I. Chalkidis, A. Jana, D. Hartung, M. Bommarito, I. Androutsopoulos, D. M. Katz, and N. Aletras, “LexGLUE: A Benchmark Dataset for Legal Language Understanding in English,” in *Proc. 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022, pp. 4310–4330.
- [14] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” in *Proc. EMNLP*, 2016, pp. 2383–2392.
- [15] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.